

Experiment 5 Interrupt Service Routines

ART
9/21/26

Purpose

Use the PIC16F883 interrupt system to observe how asynchronous hardware events change program execution. Understand the interrupt Vector & how to preserve program context.

Standards & References

- RCET Lab standards
- RCET PIC-AS style Guide
- RCET Flowchart Guide
- PIC16F883 datasheet & Family manual
- Previous lab writeups

<https://github.com/Ashton-Tolman/Semester-5-Lab-5.git>

Labs github Repo Link

Equipment & materials

- MPLAB X IDE & PIC-AS toolchain
- PICkit program debugger
- O-Scope with optional logic analyzer
- Required circuit components

Part 1 - External Interrupt & Interrupted Main Execution

Goal

Configure RBO/INT for a falling edge interrupt. Each valid interrupt increments Port C while main constantly toggles Port A

Before Lab

- Push button, PIC, & LED Schematic
- Loading analysis - ~~Same button~~
- Relevant SFRs
- External interrupt setup sequence
- Context saving
- Flow chart

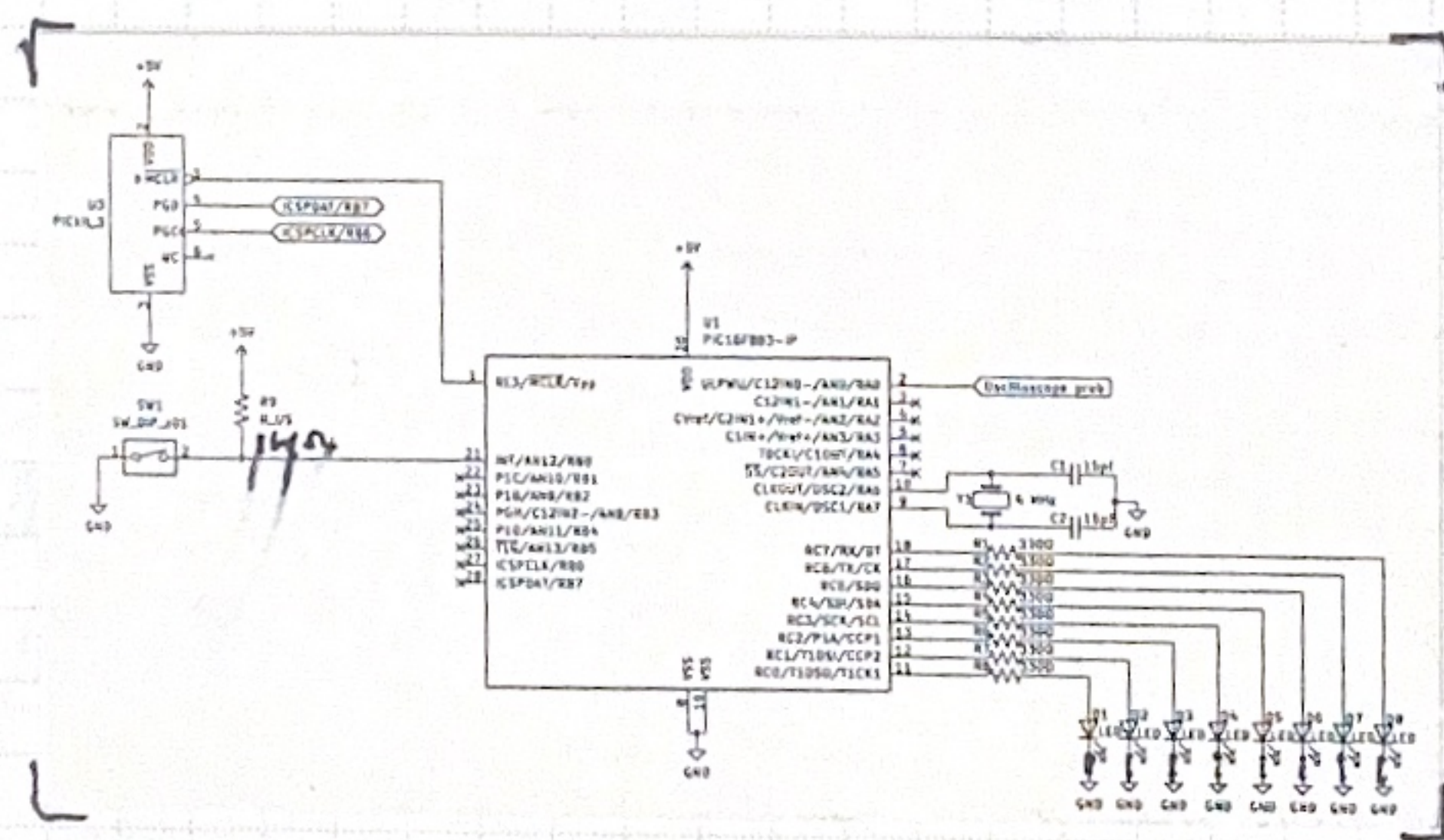


Figure 1-1
LED & External
Push button
Setup

Worst case calcs

Same as pg 156 for the push button

Output pull down resistors

$$V_{OH}/I_{pub} = 86K \leftarrow 30 \text{ is below}$$

AAT

9/22/26

Experiment 5 Interrupt Service Routines

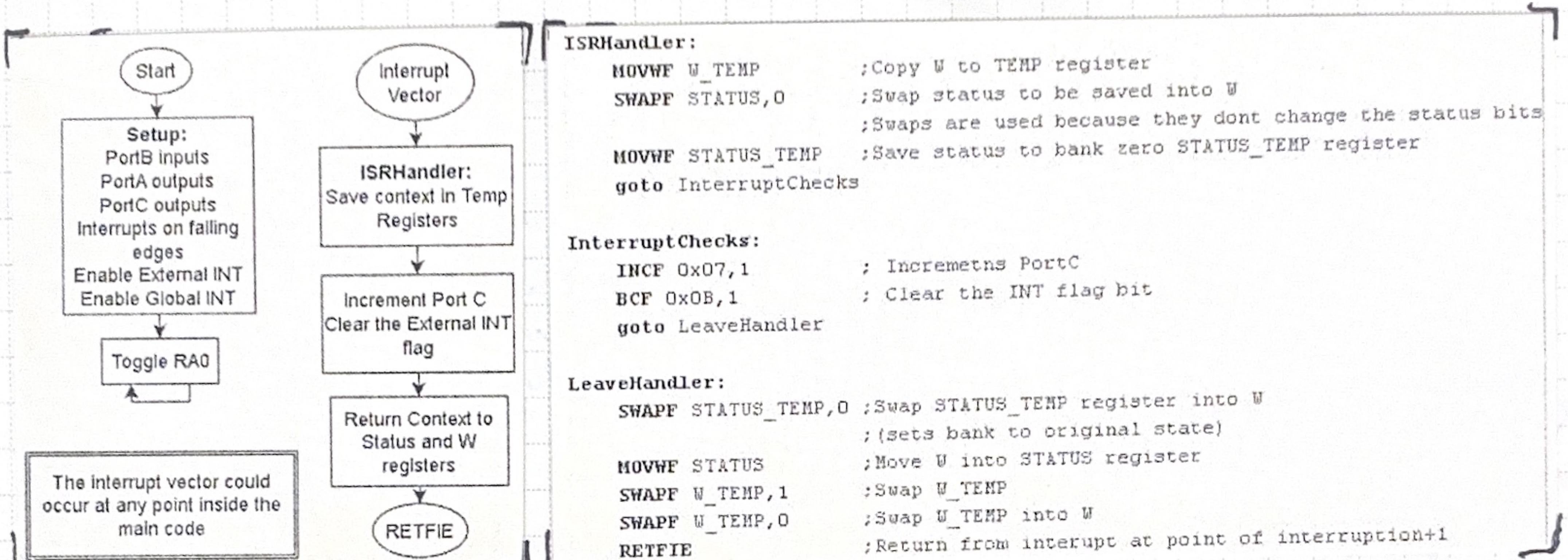
ART
9/22/26

```

MOVWF W_TEMP      ;Copy W to TEMP register
SWAPF STATUS,W    ;Swap status to be saved into W
                  ;Swaps are used because they do not affect the status bits
MOVWF STATUS_TEMP ;Save status to bank zero STATUS_TEMP register
:
: (ISR)            ;Insert user code here
:
SWAPF STATUS_TEMP,W ;Swap STATUS_TEMP register into W
                  ;(sets bank to original state)
MOVWF STATUS       ;Move W into STATUS register
SWAPF W_TEMP,F     ;Swap W_TEMP
SWAPF W_TEMP,W     ;Swap W_TEMP into W
    
```

Figure Code 1-1 Context Saving Code

* Context saving is required for interrupts because interrupts can occur at any point in the code when a W calculation is happening & the status is in a specific configuration. This must be preserved so ISR Handling doesn't break the Main code.



FC 1-1 Interrupt External INT for Code 1-2

Code 1-2 ISR Handler with applied Context Saving

* The code is Setup to Interrupt on a Falling edge on RBO/IAT, Increment all 8 bits of the PortC Register displayed on LEDs. RAO simply toggles Back & forth, Only stopping when interrupted

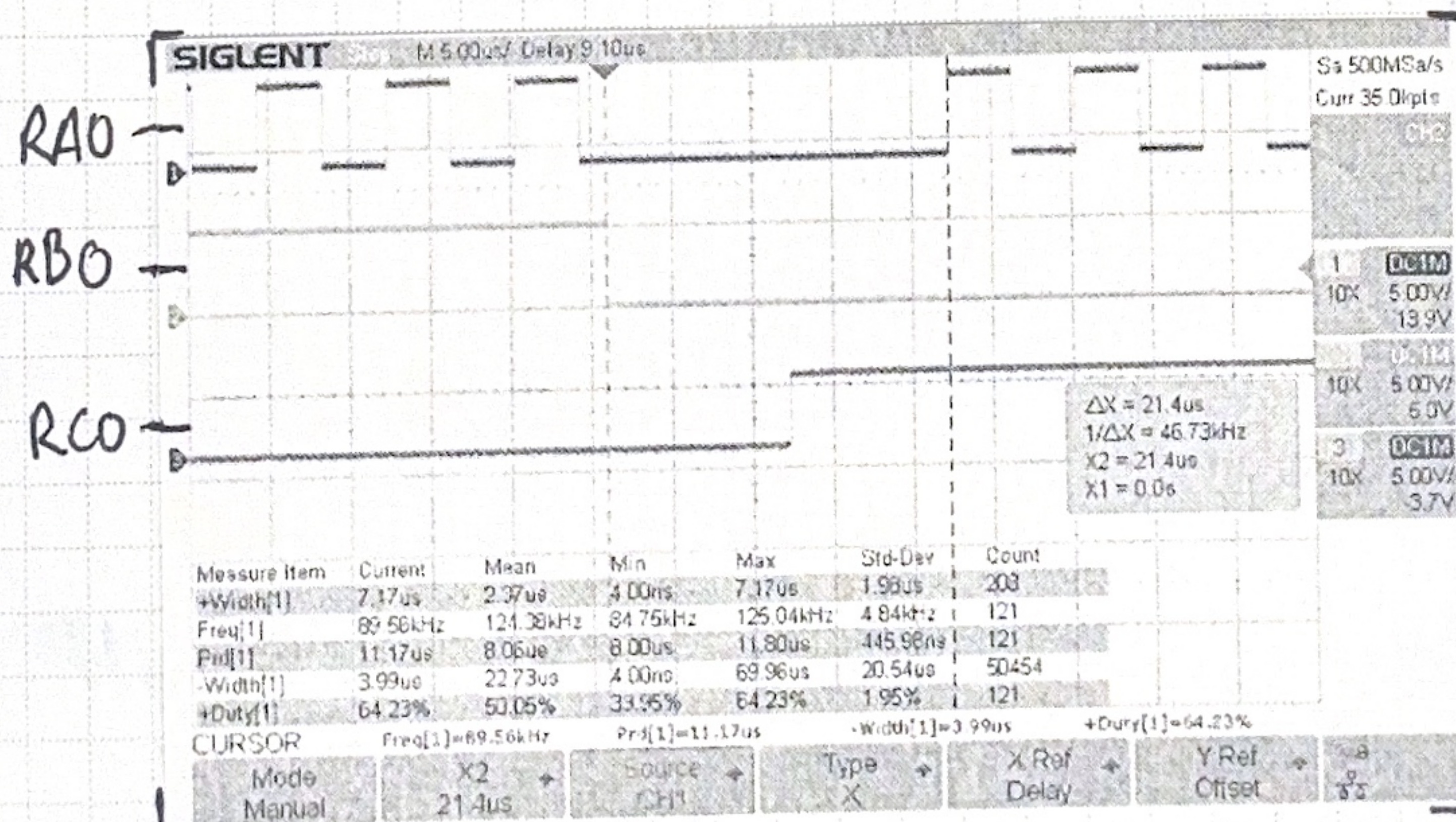


Figure 1-2 Measured RBO, RAO, RCO

* The PortC incrementing does show signs of switch bouncing that isn't visible on the o-scope

ART
9/22/26
181

Experiment 5 Interrupt Service Routines
 * Observe Code 1-2. The time it takes from when the RAO toggling stops to where it begins is calculated below:

Goto ISR Handler +2 Tcy

Context Save beginning +3 Tcy
 & Goto Interrupt checks +2 Tcy

Interrupt checks +2 Tcy
 & Goto Leave Handler +2 Tcy

Leave Handler
 & Context Save ending +4 Tcy
 & RETIE +2 Tcy

+3 Tcy for the interrupt status delay
 to notice the interrupt &
 (could be 4) go to the Vector

Total = 20 Tcy ← This lines up with the
 measured distance
 between RAO change
 (Figure 1-2) & the toggle restarting

TJE
 9-22-26

Part 2 - Two Interrupt Sources, One Interrupt Vector

Goal

Add RB4 Interrupt-on-change while retaining RB0/INT. RB4 IOC increments Port C. Emphasis is on source determination, per source service, & common exit

Before Lab

Prepare:

- Updated Schematic with button on RB4
- IOC SFR documentation
- Updated port pin assignments
- Flowchart for one vector & multiple checks
- Context Save / common exit
- Source code

Worst case cases for new circuit game as pg 180

INTCON, OPTION_REG, ON pg 311
 PORTB IOCB on pg 309 } Noted SFRs

Experiment 2 Interrupt Service Routines

ART
9/22/26

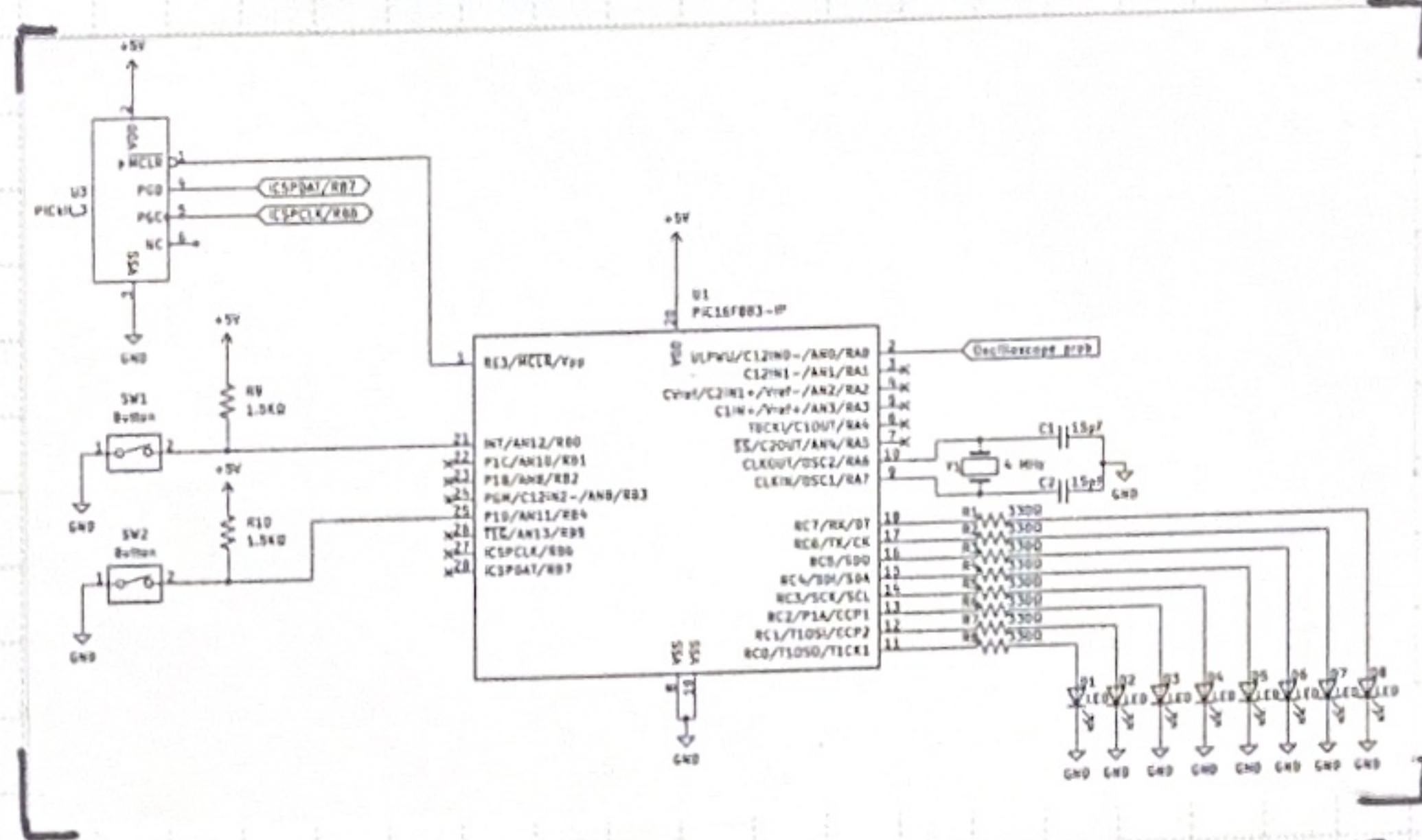


Figure 2-1 Two button External INT with diodes

* RB0 now uses the INT External to Clear PORTC. RB4 uses the PortBIOC to increment PortC. Port A RA0 is toggling in main as before (pg 180).

```

ISRHandler:
    MOVWF W_TEMP      ;Copy W to TEMP register
    SWAPF STATUS,0     ;Swap status to be saved into W
                        ;Swaps are used because they dont change the status bits
    MOVWF STATUS_TEMP ;Save status to bank zero STATUS_TEMP register

    BTFSC INTCON,1     ;Test the INT external flag
    goto INTExternalService ;if the flag is up go to service
    BTFSC INTCON,0     ;Test the IOCPortB flag
    goto IOCPORTBService ;if the flag is up go to service
    goto LeaveHandler

INTExternalService:
    CLRF 0x07          ;Clear PortC
    BCF INTCON,1       ;Clear the INTExternal flag
    goto LeaveHandler

IOCPORTBService:
    MOVF 0x06,0        ;Move 0x06 to W
    INCF 0x07,1        ;Increment PortC
    BCF INTCON,0       ;Clear the IOCPortB flag
    goto LeaveHandler

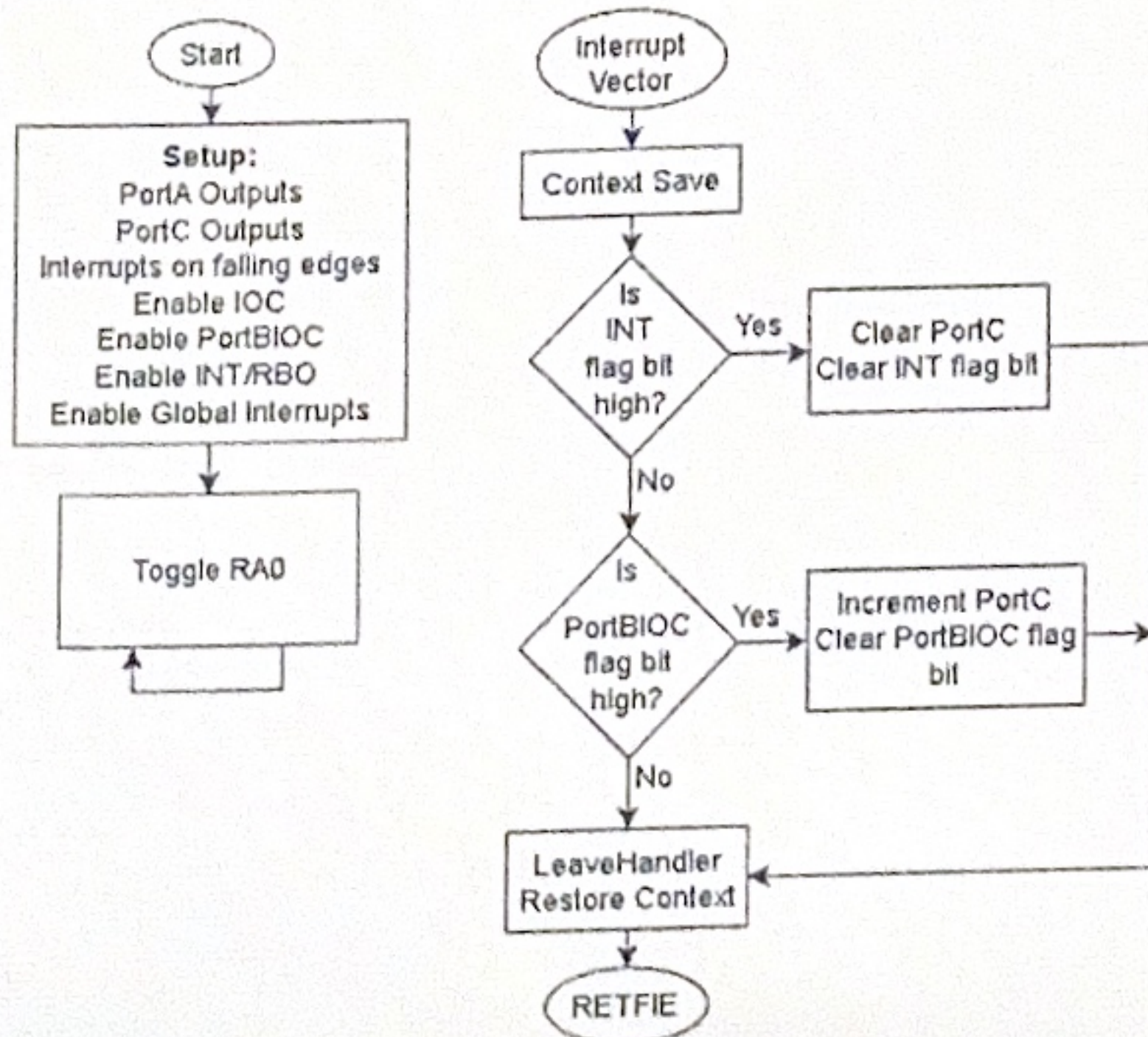
LeaveHandler:
    SWAPF STATUS_TEMP,0 ;Swap STATUS_TEMP register into W
                        ;(sets bank to original state)
    MOVWF STATUS       ;Move W into STATUS register
    SWAPF W_TEMP,1     ;Swap W_TEMP
    SWAPF W_TEMP,0     ;Swap W_TEMP into W
    RETFIE             ;Return from interrupt at point of interruption+1
    
```

Code 2-1 Multiple int Sources & Handlers

* once an Interrupt occurs the handler tests the flags possible and handles them in priority order. If an interrupt is serviced it will leave the handler before servicing another but the other's flag should still return to the handler.

* Because any portB change triggers an interrupt switch bouncing on the increment is even worse than it was before.

FC 2-1 fore for code 2-1



ART
9/23/26
183

Experiment 5 Interrupt Service Routines

ART
9/23/26

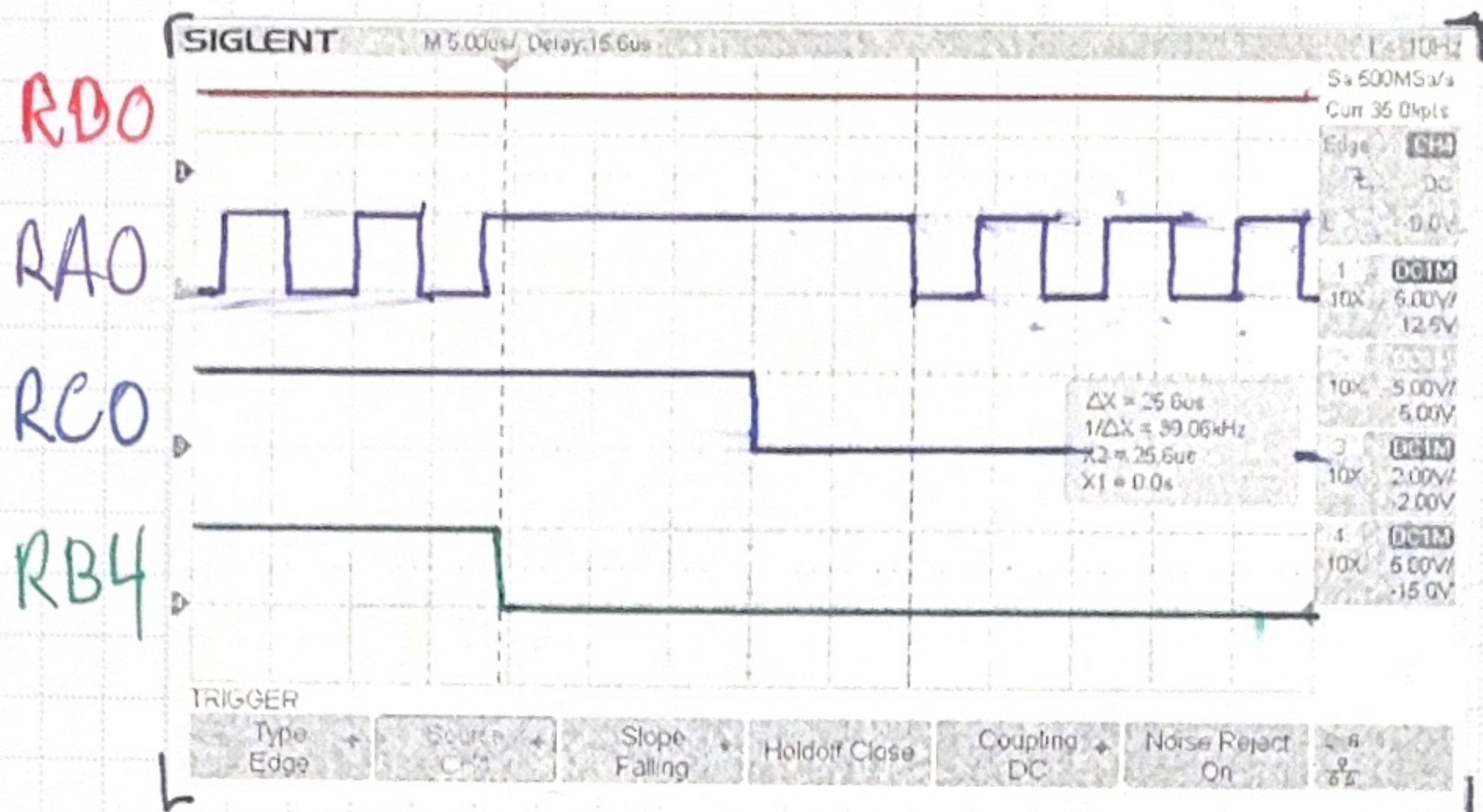


Figure 2-2 RB4 IOC incrementing Event

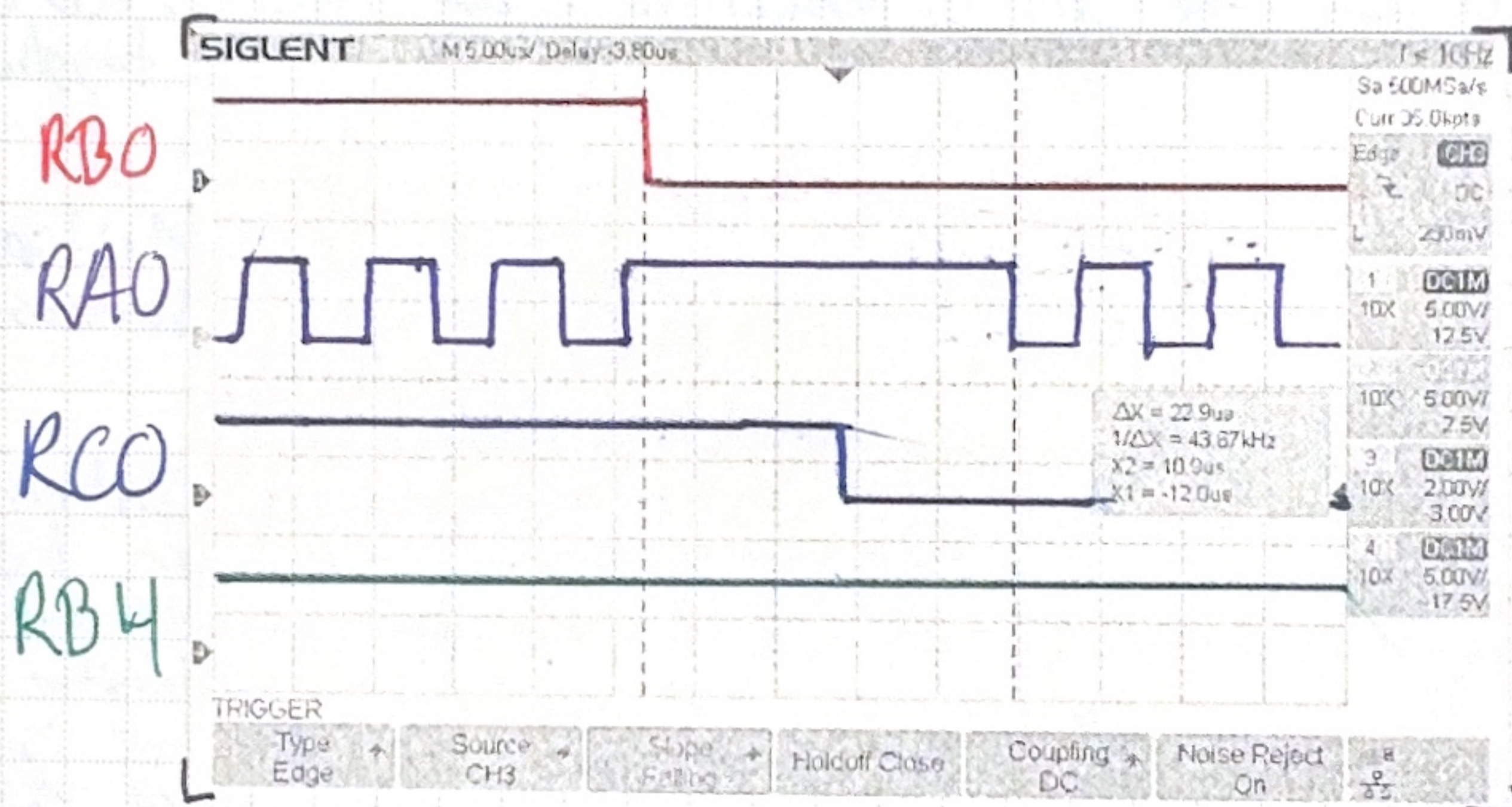


Figure 2-3 RBO INT Clear Event

```

Setup:
; add device setup here
BSF 0x03, 06 ;
BSF 0x03, 05 ; Set bank 3 active
CLRF 0x169 ; ANSELH analog/digital select portB pins as digital
CLRF 0x168 ; ANSEL analog/digital select portA pins as digital

BSF 0x03, 06 ;
BCF 0x03, 5 ; Set bank 2 active

BCF 0x03, 06 ;
BSF 0x03, 05 ; Set bank 1 active
CLRF 0x85 ; Trisa sets all portA pins as outputs
CLRF 0x87 ; Trisc sets all portC pins as outputs
BCF 0x81, 6 ; Make interrupts trigger on falling edges
BSF 0x96, 4 ; Enable IOC on bit 4

BCF 0x03, 06 ;
BCF 0x03, 05 ; Set bank 0 active
CLRF 0x05 ; Clear port A
CLRF 0x07 ; Clear port C

; Interrupt Setup
CLRF 0x0B ; Clear INTCON to disable all interrupts by default
BSF 0x8B, 3 ; Enable PortB interrupt on change
BSF 0x8B, 4 ; Enable External interrupts
BSF 0x8B, 7 ; Enable Global interrupts

BCF 0x03, 06 ;
BCF 0x03, 05 ; Set bank 0 active
goto Main
    
```

Code 2-2 Setup
for PortBIOC
& INT

TJR
9-23-26

ART
9/23/26

Part 3 - Determine Which Port B Pin changed

Goal

Use IOC PortB on all 8-bits & determine in software which pin changed.
Display the changed Pin Value of on three portA pins (000 = RB0, 111 = RB7)

Before Lab

```
ISRHandler:
    MOVWF W_TEMP      ;Copy W to TEMP register
    SWAPF STATUS,0     ;Swap status to be saved into W
                        ;Swaps are used because they dont change the status bits
    MOVWF STATUS_TEMP  ;Save status to bank zero STATUS_TEMP register

    BTFSC INTCON,0     ;Test the IOCPortB flag
    goto IOCPORTBService ;if the flag is up go to service
    goto LeaveHandler
```

```
IOCPORTBService:
    MOVWF PORTB, W
    MOVWF PB_NEW, W
    XORWF PB_OLD, W

    MOVWF PB_CHANGED
    MOVWF PB_NEW, W
    MOVWF PB_OLD
```

Call TestingTable

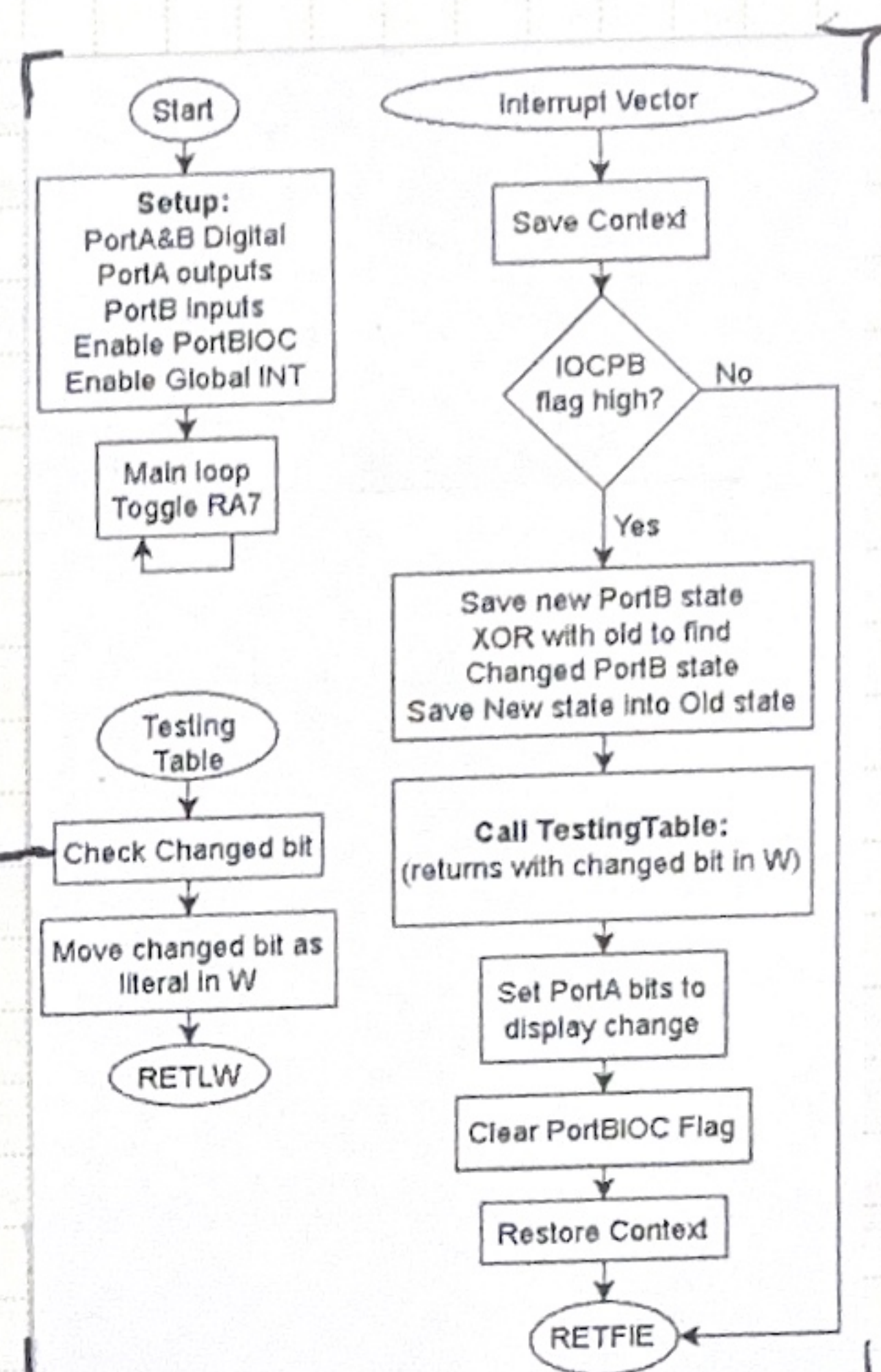
```
SetPin:
    MOVWF PORTA
    goto FinalService
```

```
TestingTable:
    BTFSC PB_CHANGED,0
    RETLW 0x00
    BTFSC PB_CHANGED,1
    RETLW 0x01
    BTFSC PB_CHANGED,2
    RETLW 0x02
    BTFSC PB_CHANGED,3
    RETLW 0x03
    BTFSC PB_CHANGED,4
    RETLW 0x04
    BTFSC PB_CHANGED,5
    RETLW 0x05
    BTFSC PB_CHANGED,6
    RETLW 0x06
    BTFSC PB_CHANGED,7
    RETLW 0x07
```

```
FinalService:
    BCF INTCON,0
    goto LeaveHandler
```

```
LeaveHandler:
    SWAPF STATUS_TEMP,0 ;Swap STATUS_TEMP register into W
                        ;(gets bank to original state)
    MOVWF STATUS        ;Move W into STATUS register
    SWAPF W_TEMP,1      ;Swap W_TEMP
    SWAPF W_TEMP,0      ;Swap W_TEMP into W
    RETFIE              ;Return from interrupt at point of interruption+1
```

uses RETLW from
Lookup Table ideas
(pg 174)



FC 3-1 for code 3-1

Code 3-1 Port B pin detection

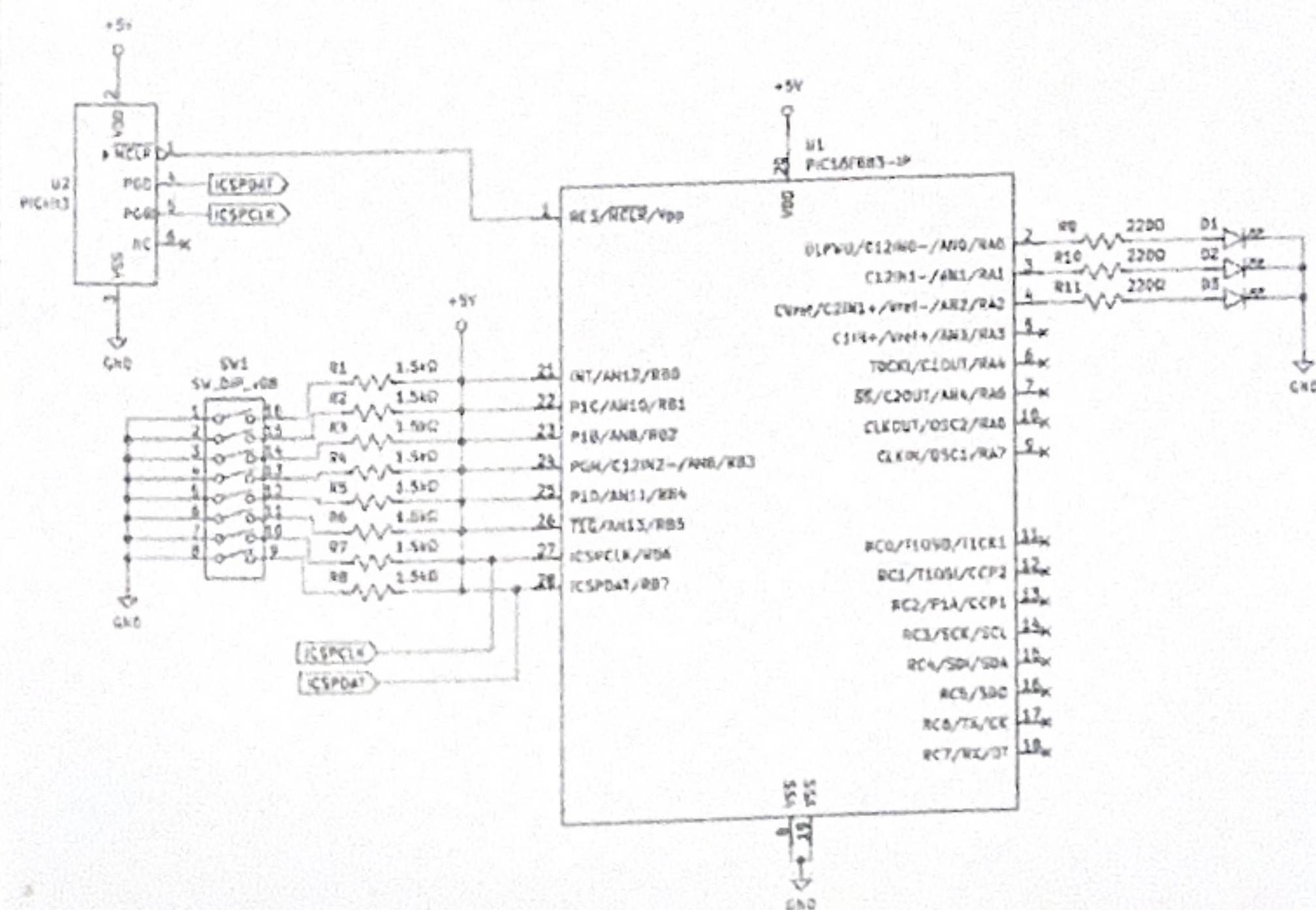


Figure 3-1 OLP on Port B
LED's on Port A

- * The function of Code 3-1 is to Interrupt RA7 toggling anytime PortB changes. The handler will then detect which pin changed & light three LEDs on PortA to show which one. RA0-2 are used for the LEDs. The code is setup with priority on RB0 but that test should only ever have one bit high in the register so there is virtually no priority.
- * There is some DIP bouncing present that causes what might look like a clear function on the RA LED's. The bounce sets the changed pin then immediately unsets it, causing the lookup to see nothing & leave the handler

In the Lab

1. Verify RB0 - RB7
2. Confirm the three bit display
3. Test repeated changes on one pin (should see no change)
4. create at least one change condition where 2 changes are present
5. Record any bounce effect
6. Verify RB6 & RB7 with the PICKit connections during normal operation.

- * I built a breakout board for my PICKit to use separately from another circuit. The data & clock pins would be overridden if normal operation was also happening & the programming would not upload.

TJR
9-24-20

ART
9/25/20

Part 4 - Priority Interrupts, Nested Execution, & Stack Depth

Goal

Create V6/7 priority-interrupt behavior using the DLG7137 display on Port C from earlier Labs. This allows 7 to interrupt 6 to investigate the nested execution

Before Lab

Prepare:

- earlier DLG7137/Port C Schematic, encoding, & components (pg 151) (pg 153) (pg 152)
- RBO external interrupt & RBI I/O wiring
- Interrupt SFR documentation (pg 311)
- Nested interrupt flowchart
- Nested context save strategy
- delay Subroutine structure
- worst case hardware stack analysis
- Source code

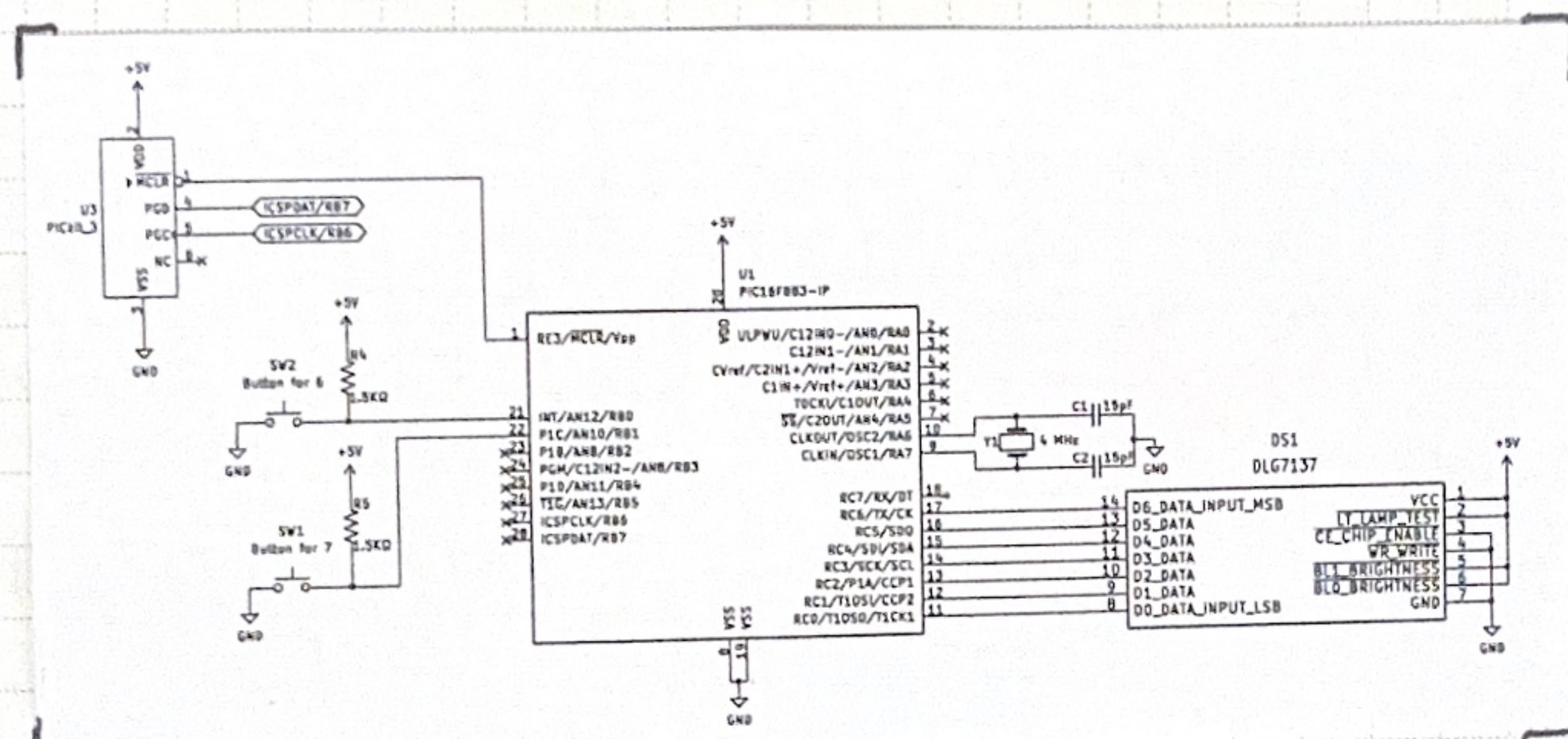


Figure 4-1 DLG7137
& two
external
buttons

Worst case calcs are same as pg 182

Hardware stack Analysis

The absolute most I push to the stack with my code is 4 Levels deep, 1 for 6 interrupt & its call delay, 1 for the 7 interrupt & 1 for its call delay. → 4.

*If the stack was pushed on more than 8 times, it would roll over, losing all return data, sending the PC into unknown no-man's-land.

```

W_TEMP_A EQU 0x70      ; Temporary W register in global GR A
STATUS_TEMP_A EQU 0x71 ; Temporary Status register in global GR A

W_TEMP_B EQU 0x72      ; Temporary W register in global GR B
STATUS_TEMP_B EQU 0x73 ; Temporary Status register in global GR B

INTDetected EQU 0x74    ; Global GR to detect 6 or 7
OldDisplay EQU 0x75     ; Old display things for 7

outCount6 EQU 0x76      ; Count for outer delay loop
inCount6 EQU 0x77       ; Count for inner delay loop
mainCount6 EQU 0x78     ; Count for main delay loop
outCount7 EQU 0x79      ; Count for outer delay loop
inCount7 EQU 0x7A       ; Count for inner delay loop
mainCount7 EQU 0x7B     ; Count for main delay loop
  
```

Code 4-2

Global Registers used
in code 4-1

Experiment 3 Interrupt Service Routines

ART
9/25/26

```
ISRHandler:
    BTFSC INTCON,0      ;IOC flag test
    goto Context7       ;Save context for a 7 interrupt
                        ;If not 7 save context for 6 interrupt

Context6:
    MOVWF W_TEMP_A      ;Copy W to TEMP register
    SWAPF STATUS,0       ;Swap status to be saved into W
                        ;Swaps are used because they dont change the status bits
    MOVWF STATUS_TEMP_A  ;Save status to bank zero STATUS_TEMP register
    goto INTExternalService ;if the flag is up go to service

Context7:
    MOVWF W_TEMP_B      ;Copy W to TEMP register
    SWAPF STATUS,0       ;Swap status to be saved into W
                        ;Swaps are used because they dont change the status bits
    MOVWF STATUS_TEMP_B  ;Save status to bank zero STATUS_TEMP register
    goto IOCPORTBService ;if the flag is up go to service

INTExternalService:
    BCF INTCON,1         ;Clear INT flag
    BCF INTCON,4         ;Turn INT off so 6 cant interrupt 6
    BSF INTCON,7         ;Turn Global interrupts on

    MOVLW 0x36           ;Put display code for 6 in W
    MOVWF 0x07           ;Display 6 on PortC

    CALL Delay_2Sec6     ;Call 2 second delay
    BSF INTDetected,0    ;If it was a 6 INT, set GP bit to indicate

    goto LeaveHandler

IOCPORTBService:
    BCF INTCON,7         ;ReEstablish Globals off so 7 cant be interrupted
    MOVF PORTB,W         ;Dummy check PortB for IOC
    BTFSC PORTB,1        ;Test RB1 to make sure it cant trigger on a rising edge
    goto IOC_Ignore      ;If RB1 is high 7 will not run for 2 more seconds
    MOVF PORTC,W         ;Put current display state in W
    MOVWF OldDisplay     ;Store old display state

    MOVLW 0x37           ;Move display code for 7 in W
    MOVWF 0x07           ;Display 7 on PortC

    CALL Delay_2Sec7     ;Call 2 second delay

    MOVF OldDisplay,W    ;After 7 move previous display to W
    MOVWF PORTC          ;Display previous state

    IOC_Ignore:
    BCF INTCON,0         ;Clear IOC flag
    goto LeaveHandler

LeaveHandler:
    BTFSS INTDetected,0  ;IOC flag test
    goto LeaveHandler7   ;Load context for a 7 interrupt
                        ;If not 7 Load context for 6 interrupt

LeaveHandler6:
    BCF INTDetected,0    ;Clear the is 6 flag in GP
    BSF INTCON,4         ;Turn on INT so 6 can fire again if needed
    SWAPF STATUS_TEMP_A,0 ;Swap STATUS_TEMP register into W
                        ;(sets bank to original state)
    MOVWF STATUS         ;Move W into STATUS register
    SWAPF W_TEMP_A,1     ;Swap W_TEMP
    SWAPF W_TEMP_A,0     ;Swap W_TEMP into W
    RETFIE               ;Return from interrupt at point of interruption+1

LeaveHandler7:
    SWAPF STATUS_TEMP_B,0 ;Swap STATUS_TEMP register into W
                        ;(sets bank to original state)
    MOVWF STATUS         ;Move W into STATUS register
    SWAPF W_TEMP_B,1     ;Swap W_TEMP
    SWAPF W_TEMP_B,0     ;Swap W_TEMP into W
    RETFIE               ;Return from interrupt at point of interruption+1
```

} check INTCON flags for 6 or 7
So 7 & 6 can interrupt 7

} Save context for a 6 interrupt.

} Save context for a 7 interrupt

} Turn Global INT off on
So 7 can interrupt 6

} Display 6 & wait 2 sec

} Set the 6 flag for context load

} If 7 interrupts anything
turn global Int off &
test RB1 for a high so
it only initiates on falling
Display 7 & wait 2 sec

} Old display is saved &
returned here

} Clear IOC flag

} Recall 6 flag to decide
to load 6 or 7

} ReEnable the INT so 6
can be pressed again.
Clear the 6 flag
Recall 6 Context Save

} Recall 7 context Save

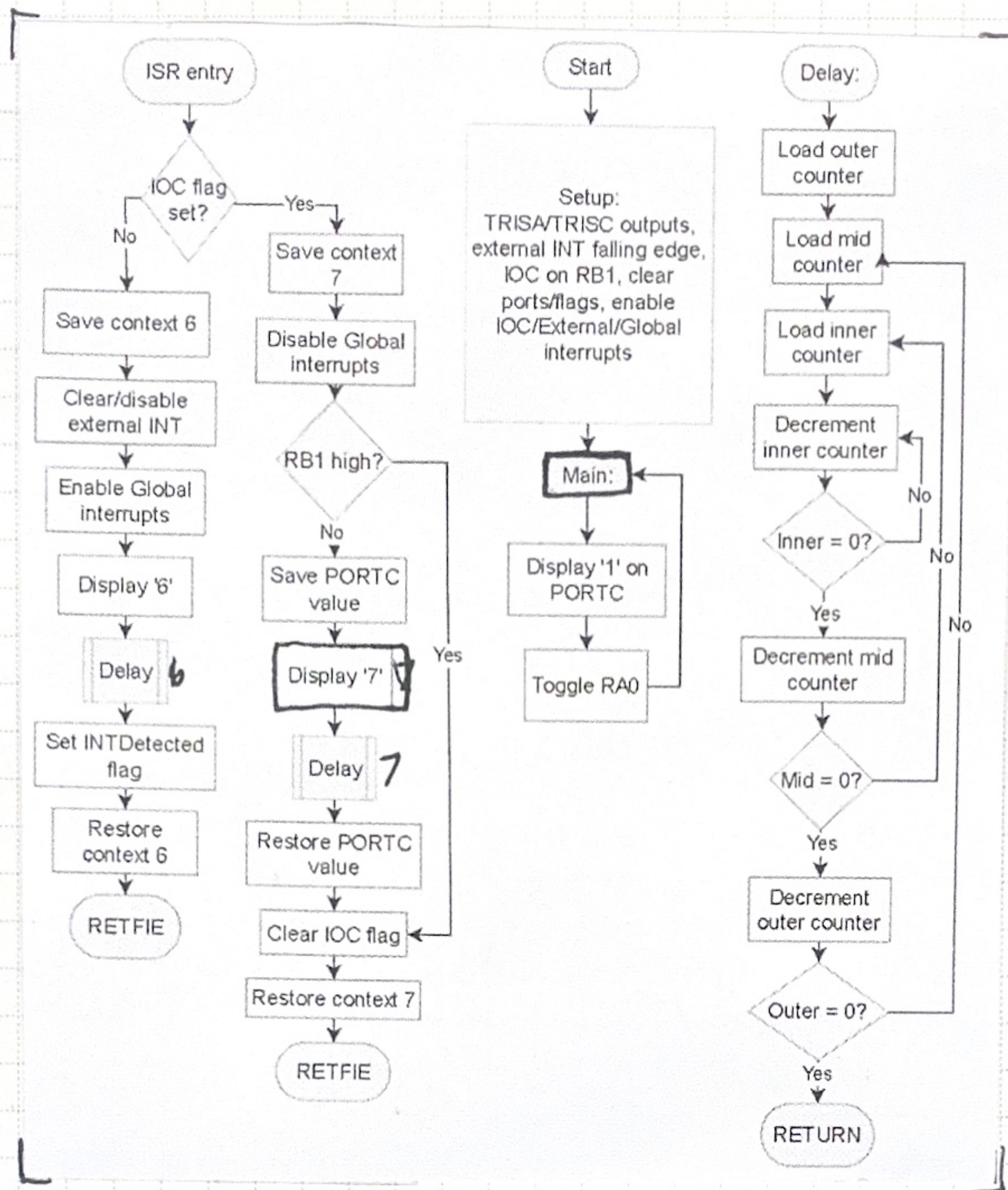
Code 4-1 Nested Interrupts

* The only addition not shown here that is necessary is there is a 7 delay & a 6 delay. If both interrupts shared a delay loop, 6 would restart when interrupted or roll over to the highest possible count numbers

ART
9/25/26

Experiment 5 Interrupt Service Routine

ART
9/25/26



FC4-1 for code 4-1

Conclusion

This lab served to measure & understand Interrupts, their flags, timing, & service sequence. The important notes to keep are context saving, which save the state of the status Register & Wg as well as the program stack. Remember to keep track of the various ways Interrupts can occur as well as the flags used to detect them. Other than that the interrupt code sequence follows normal processes same as main.

ART
9/25/26